

CopilotKit x Deep Agents

Engineering Issues Report

deep-agents-finance-erp showcase | April 2026

Summary

Four platform-level issues were identified during the build of the CopilotKit + LangChain deep agents finance ERP showcase. These affect the Python SDK, middleware event pipeline, and React SDK when used with multi-agent orchestration (**create_deep_agent**). All have local workarounds in place, but need upstream fixes to work correctly out of the box.

#	Issue	Severity	Status
01	copilotkit.actions empty in agent traces	High	Worked around
02	emit_tool_calls filtering broken for list/string	High	Fixed locally
05	Metadata key mismatch on subgraph events	Medium	Fixed locally
08	useFrontendTool incompatible with deep agents	High	Workaround

#01 — *copilotkit.actions is empty in agent traces*

Severity: High | Component: Python SDK — CopilotKitMiddleware / langgraph_agui_agent

Problem: The **copilotkit.actions** context object (the runtime-injected list of frontend tools) is consistently empty when the LangGraph agent executes. The middleware cannot identify which tool calls are frontend tools, so interception silently fails. Frontend tool calls execute as backend stubs returning plain strings instead of streaming UI components.

Workaround: Pass `frontend_tool_names` explicitly to the middleware and to `create_deep_agent(tools=...)` so tools are available without relying on `copilotkit.actions`.

Questions for engineering:

- Why does `copilotkit.actions` not populate when the runtime forwards frontend tool definitions to the Python agent?
- Is this specific to the `create_deep_agent + CopilotKitMiddleware` path, or does it also affect `CopilotKitAgent`?
- Is there a race condition where the agent graph starts executing before the runtime finishes injecting actions?

#02 — *emit_tool_calls* filtering broken for list/string values

Severity: High | Component: Python SDK — `_dispatch_event` in `langgraph_agui_agent.py`

Problem: The `emit_tool_calls` config was only checked as a boolean (is **False**). When set to a list of tool names or a string, the filtering logic treated it as truthy and emitted all tool call events, including internal tools like **task** whose raw JSON leaked into the chat stream. Additionally, `TOOL_CALL_RESULT` was not included in the filter, and subgraph events lacked the `copilotkit:emit-tool-calls` metadata key.

Local fix (commit 64d360e): Implemented proper list/string type checking, added `TOOL_CALL_RESULT` to filtered event types, and added fallback to `self.config` metadata when event-level metadata is missing.

Questions for engineering:

- Should `emit_tool_calls` be documented as supporting `bool` | `list[str]` | `str`? The type signature may only declare `bool`.
- Should subgraph events inherit parent graph metadata by default?

#05 — *Metadata key mismatch between base agent and SDK consumer*

Severity: Medium | Component: Python SDK — `metadata handling` in `_dispatch_event`

Problem: Two related metadata issues caused tool call events from subgraphs to bypass filtering. First, the base agent sets metadata as **`copilotkit:emit-tool-calls`** but the SDK dispatcher reads **`emit-tool-calls`** (without the `copilotkit:` prefix). Second, subgraph events don't carry parent metadata, so the filtering logic can't find the key at all.

Local fix (commit 64d360e): Added fallback to `self.config` metadata and normalized key lookup to handle both prefixed and unprefixed forms.

Questions for engineering:

- Standardize metadata key naming: always use `copilotkit:` prefix or never. Document the convention.
- Propagate parent graph metadata to subgraph events in LangGraph's event stream.
- Add SDK-level tests for metadata resolution on subgraph events specifically.

#08 — *useFrontendTool incompatible with deep agents*

Severity: High | Component: React SDK + Python SDK middleware + LangGraph deep agents

Problem: `useFrontendTool` relies on a middleware interception cycle: pause the graph, stream a `TOOL_CALL` to the frontend, execute the handler, return the result, resume the graph. With deep agents, the orchestrator delegates to subagents via a nested **task** tool. When the middleware pauses the outer graph, the subagent state and graph continuation don't re-enter cleanly. Tools render momentarily during streaming, then vanish when the graph finalizes.

Workaround: Switch non-HITL tools from `useFrontendTool` to **`useRenderTool`**. Backend stubs execute normally, frontend renders from tool call args (no middleware interception needed). HITL tools using `copilotkit_interrupt()` are unaffected.

Questions for engineering:

- Is `useFrontendTool` intended to work with multi-agent / deep agent graphs? If not, document this limitation.
- If it should work, the middleware pause/resume mechanism needs to handle nested subgraph execution.
- Consider recommending `useRenderTool` as the default for generative UI, with `useFrontendTool` reserved for cases where the frontend must compute a result the agent needs.